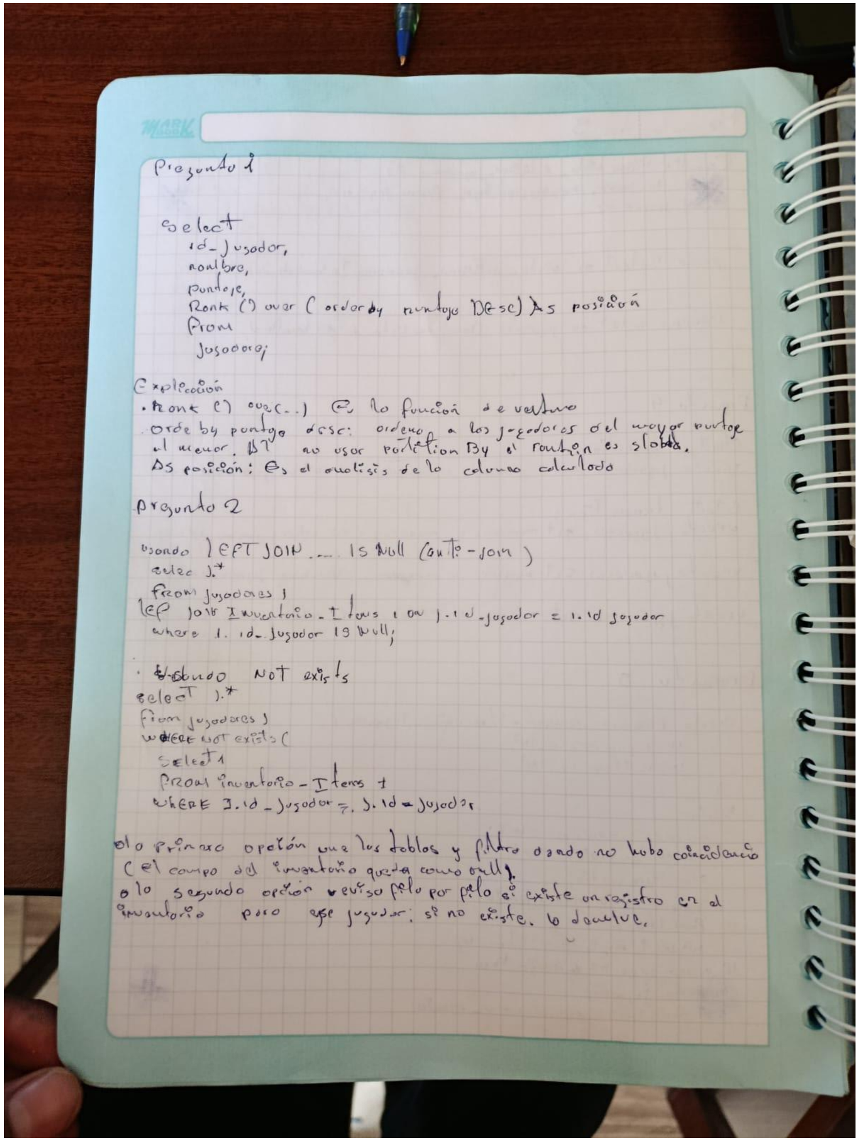


Pregunta 1

```
select
   id_jugador,
   nombre,
   puntaje,
   Rank () over ( order by puntaje DESC) As posición
   From
    jugadores;
```

Explicación
- Rank () over(...) Es la función de ventana
- orde by puntaje desc: ordena a los jugadores del mayor puntaje al menor. Al no usar partition By el ranking es global.
- As posición: Es el analisis de la columna calculada

Pregunta 2

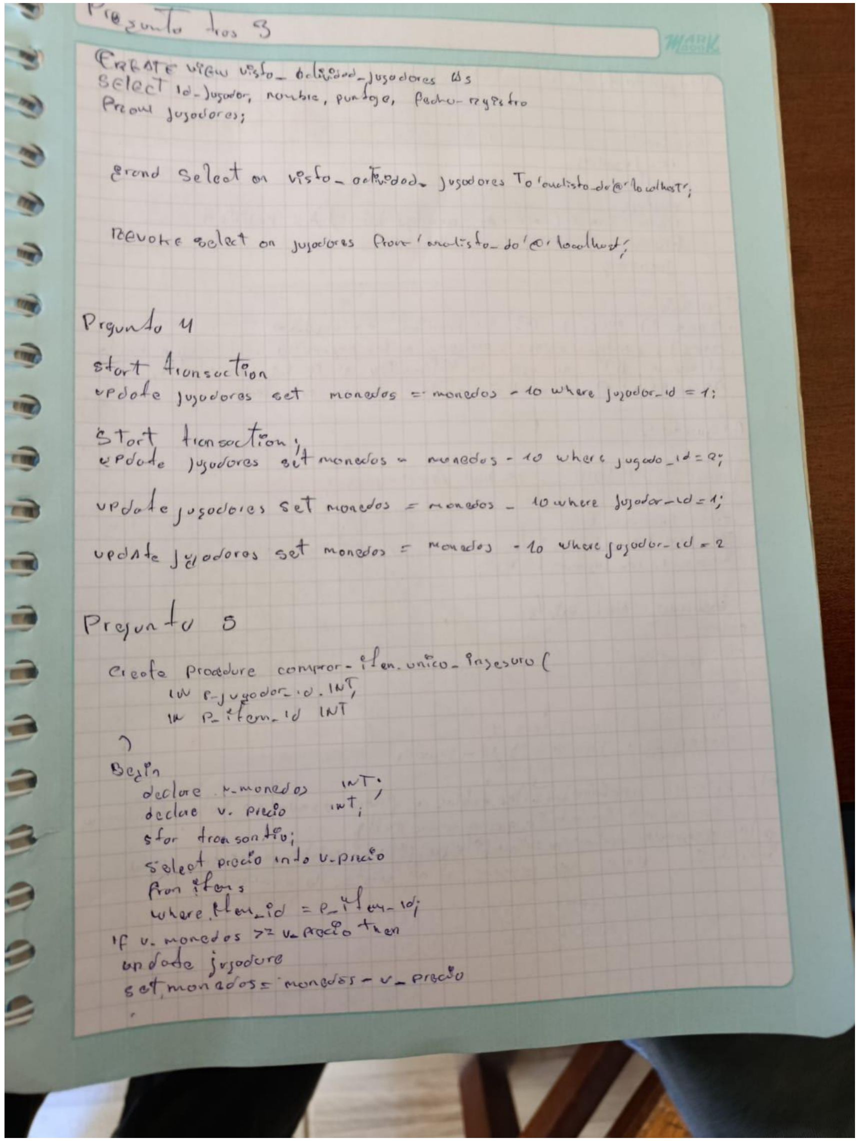
usando LEFT JOIN ... IS Null (anti-join)

```
select j.*
from jugadores j
lef join Inventario_Items i on j.id_jugador = i.id jugador
where i. id_jugador is Null;
```

· usando NOT exists

```
select j.*
from jugadores j
where not exists (
  select 1
  From inventario_Items i
  WHERE I.id_jugador =. j.id_jugador
```

- la primera opción une las tablas y filtra cuando no hubo coincidencia (el campo del inventario queda como null).
- la segunda opción revisa fila por fila si existe un registro en el inventario para ese jugador; si no existe, lo devuelve.



Pregunta dos 3

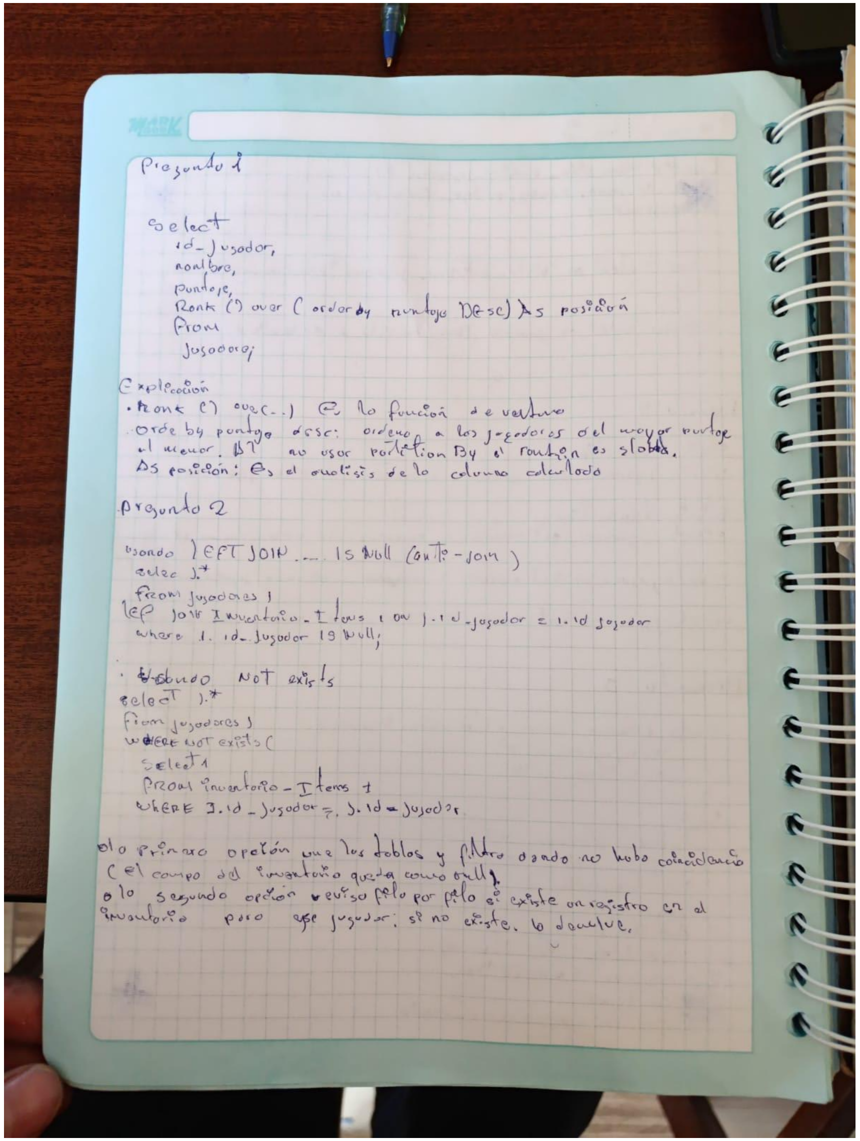
```
CREATE view vista_actividad_jugadores As
SELECT id_jugador, nombre, puntaje, fecha_registro
From jugadores;


grand Select on vista_actividad_jugadores To 'analista_de'@'localhost';


Revoke select on jugadores from 'analista_da'@'localhost';
```

Pregunta 4

```
start transaction
update jugadores set   monedas = monedas - 10 where jugador_id = 1;

Start transaction;
update jugadores set monedas = monedas - 10 where jugado_id = 2;

update jugadores set monedas = monedas - 10 where jugador_id = 1;

update jugadores set monedas = monedas - 10 where jugador_id = 2
```

Pregunta 5

```
create procedure comprar_item_unico_inseguro (
      IN p_jugador_id INT,
      IN p_item_id INT
 )
 Begin
   declare v_monedas  INT;
   declare v. precio  int;
   star transantio;
   select precio into v_precio
   from items
   where item_id = p_item_id;
 if v. monedas >= v_precio then
  update jugadore
  set monedos = monedas - v_precio
```



Pregunta 1

```
select
   id_jugador,
   nombre,
   puntaje,
   Rank () over ( order by puntaje DEsc) As posición
   From
    Jugadores;
```

Explicación
- Rank () over(..) Es la función de ventana
- Orde by puntaje desc: ordena a los jugadores del mayor puntaje al menor. Al no usar partition By el ranking es global.
- As posición: Es el analisis de la columna calculado

Pregunta 2

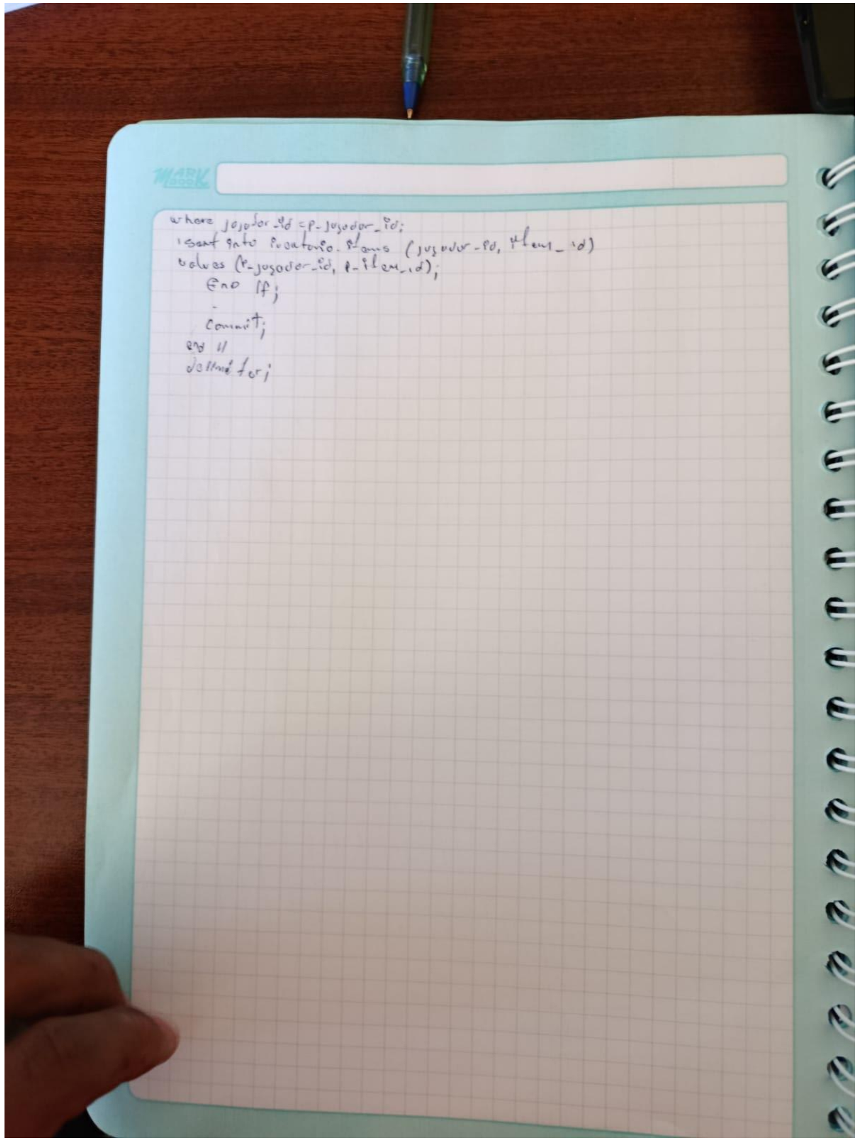
usando LEFT JOIN ... IS Null (anti-join)

```
 selec J.*
 From Jugadores J
lef Join Inventario_Items I on J.id_jugador = I.id jugador
 where I. id_Jugador IS Null;
```

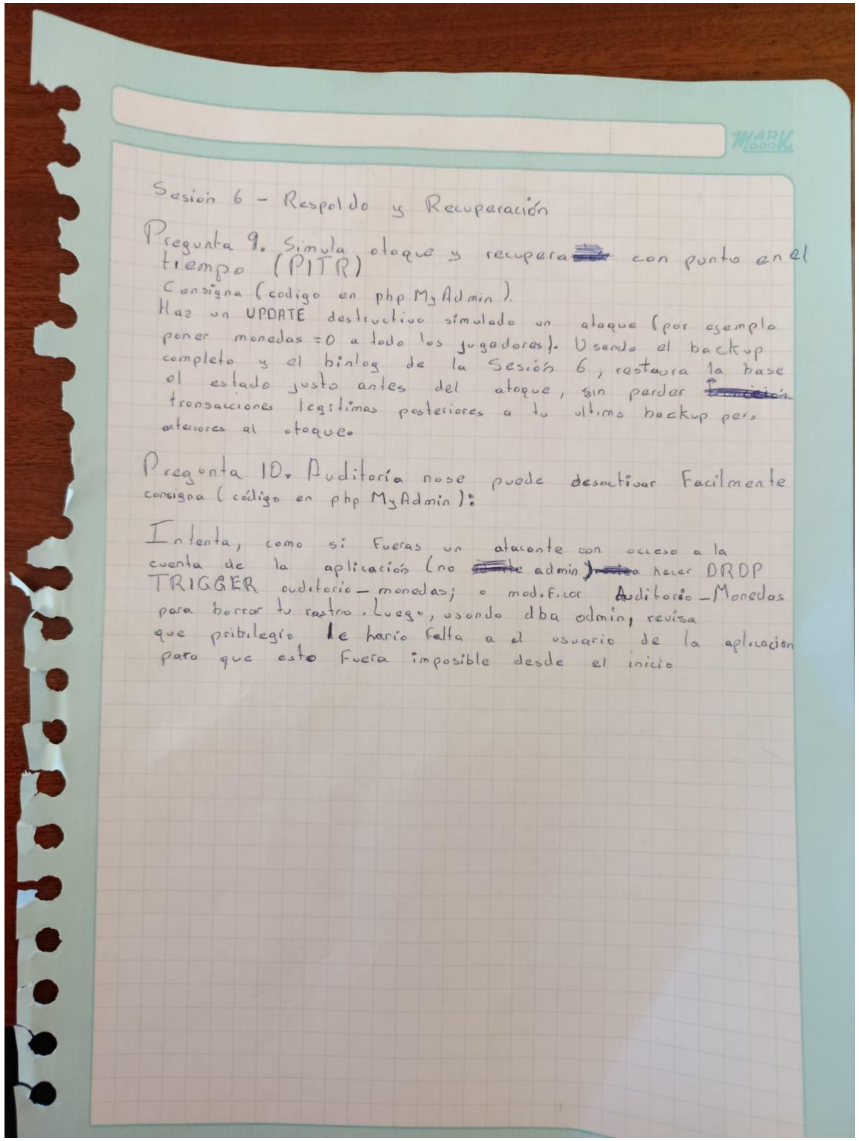
· Usando NOT exists

```
select J.*
from jugadores J
wHERE NOT exists (
  select 1
  FROM inventario_Items I
  WHERE I.id_Jugador =, J.id_Jugador
```

- la primera opción une las tablas y filtra cuando no hubo coincidencia (el campo del inventario queda como null).
- la segunda opción revisa fila por fila si existe un registro en el inventario para ese jugador; si no existe, lo devuelve.



```
where jugador_id =p_jugador_id;
isert into inventario_items (jugador_id, item_id)
values (p_jugador_id, p_item_id);
   End If;

   Commit;
end //
delimiter;
```



Sesión 6 - Respoldo y Recuperación

Pregunta 9. Simula ataque y recupera con punto en el tiempo (PITR)

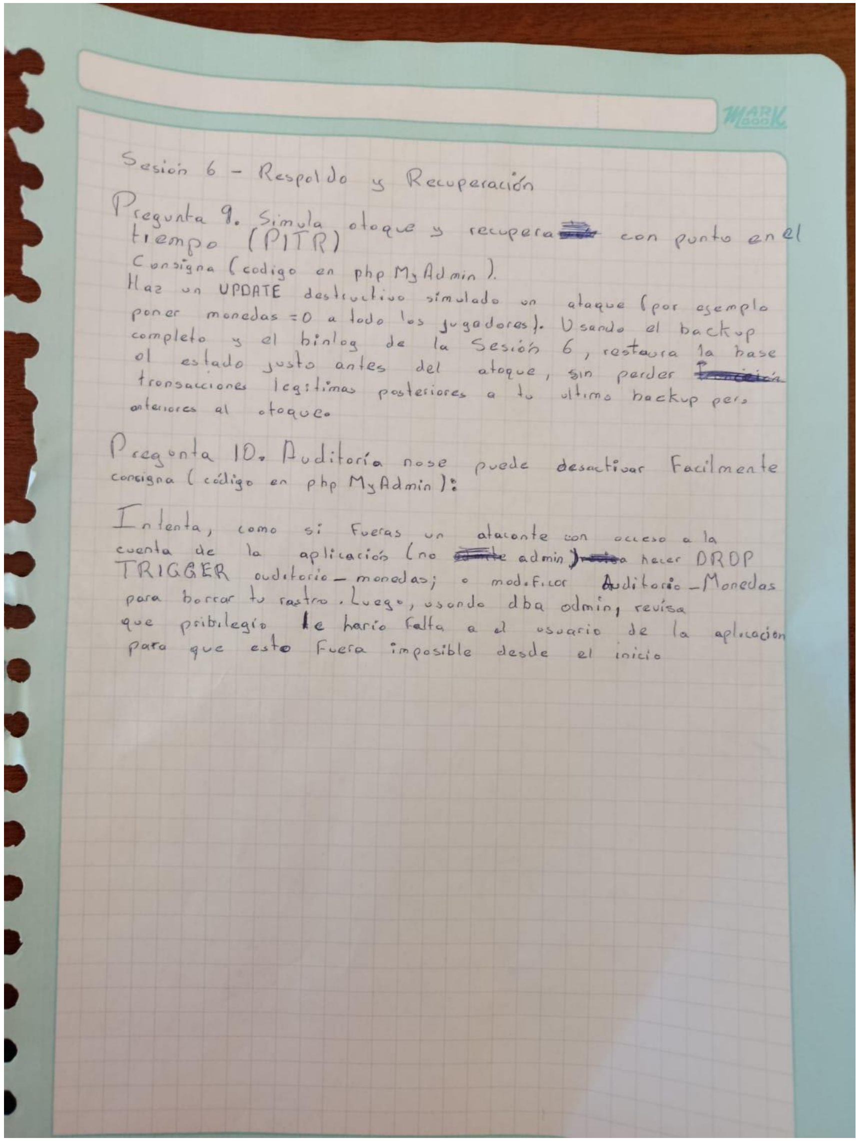
Consigna (codigo en php MyAdmin).

Haz un UPDATE destructivo simulado un ataque (por esemplo poner monedas =0 a todo los jugadores). Usando el backup completo y el binlog de la Sesión 6, restaura la base al estado justo antes del ataque, sin perder transacciones legitimas posteriores a tu ultimo backup pero anteriores al ataque.

Pregunta 10. Auditoría nose puede desactivar Facilmente

consigna (código en php MyAdmin):

Intenta, como si Fueras un atacante con acceso a la cuenta de la aplicación (no admin) a hacer DROP TRIGGER auditorio_monedas; o modificar Auditorio_Monedas para borrar tu rastro. Luego, usando dba admin, revisa que pribilegio le haría falta a el usuario de la aplicacion para que esto Fuera imposible desde el inicio



Sesión 6 - Respoldo y Recuperación

Pregunta 9. Simula ataque y recupera con punto en el tiempo (PITR)

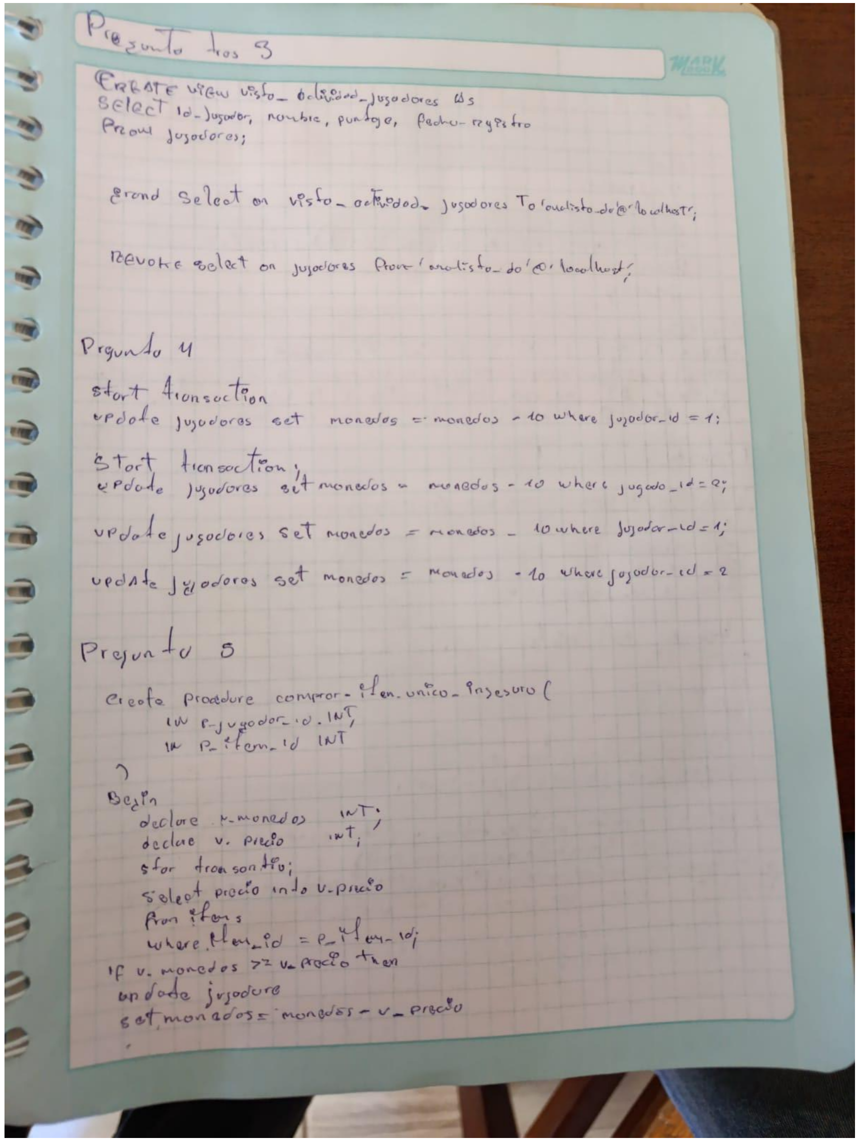
Consigna (codigo en php MyAdmin).

Haz un UPDATE destructivo simulado un ataque (por esemplo poner monedas =0 a todo los jugadores). Usando el backup completo y el binlog de la Sesión 6, restaura la base al estado justo antes del ataque, sin perder transacciones legitimas posteriores a tu ultimo backup pero anteriores al ataque.

Pregunta 10. Auditoría nose puede desactivar Facilmente

consigna (código en php MyAdmin):

Intenta, como si Fueras un atacante con acceso a la cuenta de la aplicación (no admin) a hacer DROP TRIGGER auditorio_monedas; o modificar Auditorio_Monedas para borrar tu rastro. Luego, usando dba admin, revisa que pribilegio le haría falta a el usuario de la aplicacion para que esto Fuera imposible desde el inicio



Pregunta tres 3

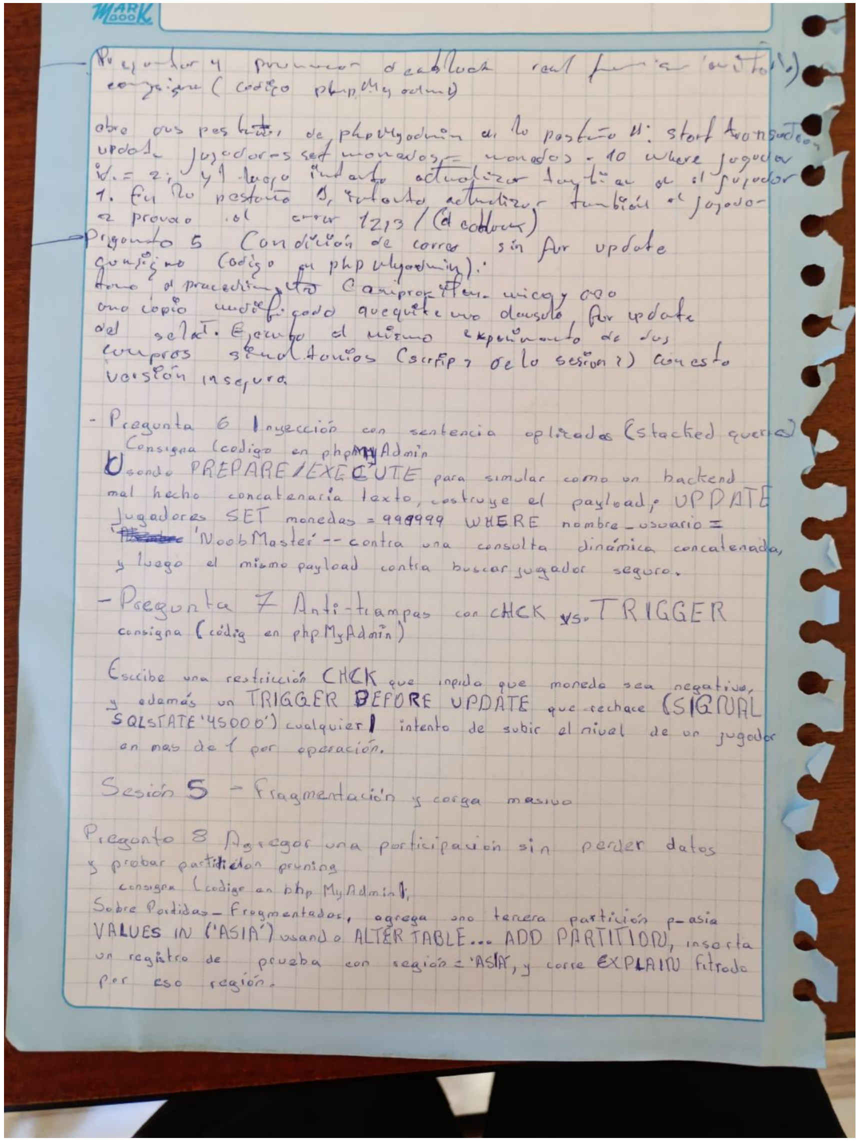
```
CREATE view vista_actividad_jugadores as
select id_jugador, nombre, puntaje, fecha_registro
from jugadores;

grant select on vista_actividad_jugadores To 'analista_datos'@'localhost';

Revoke select on jugadores from 'analista_datos'@'localhost';
```

Pregunta 4

```
start transaction
update jugadores set monedas = monedas - 10 where jugador_id = 1;

start transaction;
update jugadores set monedas = monedas - 10 where jugador_id = 2;

update jugadores set monedas = monedas - 10 where jugador_id = 1;

update jugadores set monedas = monedas - 10 where jugador_id = 2
```

Pregunta 5

```
create procedure comprar_item_unico_inseguro (
        IN p_jugador_id INT,
        IN p_item_id INT
  )
Begin
    declare v_monedas    INT;
    declare v_precio     int;
    start transaction;
    select precio into v_precio
    from items
    where item_id = p_item_id;
if v_monedas >= v_precio then
  update jugadores
  set monedas = monedas - v_precio
```



Pregunta 4 provocar deadlock real [illegible]
consigna (código phpMyadmin)

abre dos pestañas de phpMyadmin en la pestaña A: start transaction; update jugadores set monedas = monedas - 10 where jugador_id = 2; y luego intenta actualizar también al jugador 1. En la pestaña B, intenta actualizar también el jugador 2 provoca el error 1213 (deadlock)

Pregunta 5 Condición de carrera sin for update
consigna (código en phpMyadmin):
Toma el procedimiento comprar_item_unico y crea una copia modificada que quite una cláusula for update del select. Ejecuta el mismo experimento de dos compras simultáneas (script o de la sesión 2) con esta versión insegura.

- Pregunta 6 Inyección con sentencia apiladas (stacked queries)
  Consigna (codigo en phpMyAdmin

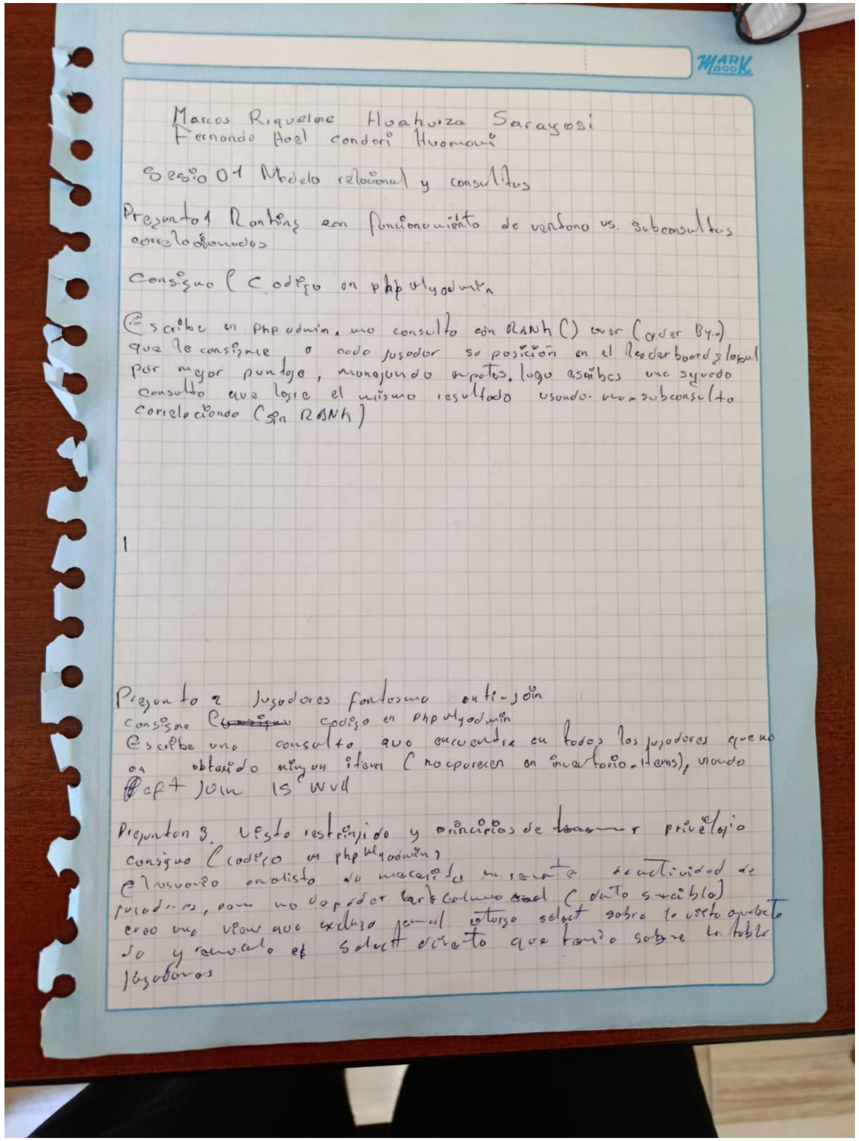
Usando PREPARE/EXECUTE para simular como un backend mal hecho concatenaria texto, costruye el payload; UPDATE Jugadores SET monedas = 999999 WHERE nombre_usuario = ~~[illegible]~~ 'NoobMaster' -- contra una consulta dinámica concatenada, y luego el mismo payload contra buscar jugador seguro.

- Pregunta 7 Anti-trampas con CHCK vs. TRIGGER
  consigna (códig en phpMyAdmin)

Escribe una restricción CHCK que impida que moneda sea negativo, y además un TRIGGER BEFORE UPDATE que rechace (SIGNAL SQLSTATE '45000') cualquier intento de subir el nivel de un jugador en mas de 1 por operación.

Sesión 5 - Fragmentación y carga masiva

Pregunta 8 Agregar una participación sin perder datos y probar partition pruning
consigna (codigo en php MyAdmin),
Sobre Partidas_Fragmentadas, agrega una tercera partición p_asia VALUES IN ('ASIA') usando ALTER TABLE... ADD PARTITION, inserta un registro de prueba con región = 'ASIA', y corre EXPLAIN filtrado por esa región.



Marcos Riquelme Huahuiza Sarayasi
Fernando Abel Condori Huamani

Sesión 01 Modelo relacional y consultas

Pregunta 1 Ranking con funcionamiento de ventana vs. subconsultas correlacionadas

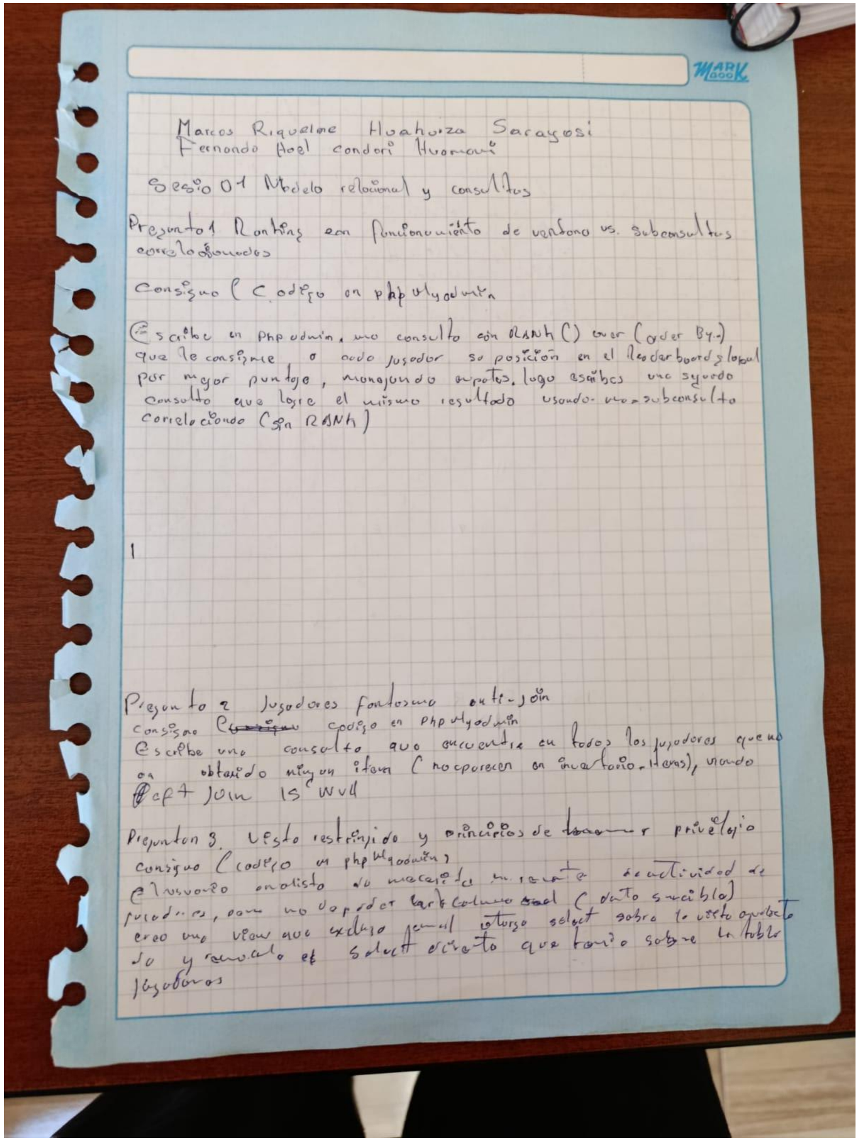
Consigna (Código en phpMyAdmin

Escribe en phpMyAdmin, una consulta con RANK() over (order By..) que le asigne a cada jugador su posición en el leaderboard global por mayor puntaje, manejando empates. Luego escribe una segunda consulta que logre el mismo resultado usando una subconsulta correlacionada (sin RANK)

1

Pregunta 2 Jugadores fantasma anti-join
Consigna (~~[illegible]~~ código en phpMyAdmin
Escribe una consulta que encuentre todos los jugadores que no han obtenido ningún item (no aparecen en inventario_items), usando left join is null

Pregunta 3. Vista restringida y principios de ~~[illegible]~~ menor privilegio
Consigna (código en phpMyAdmin)
El usuario analista necesita monitorear la actividad de jugadores, pero no debe poder ver la columna ~~[illegible]~~ (dato sensible)
Crea una view que excluya email, otorga select sobre la vista a analista y revoca el select directo que tenía sobre la tabla jugadores



Marcos Riquelme Huahuiza Sarayasi
Fernando Abel Condori Huamani

Sesión 01 Modelo relacional y consultas

Pregunta 1 Ranking con funcionamiento de ventana vs. subconsultas correlacionadas

Consigna (Código en phpMyAdmin

Escribe en phpMyAdmin, una consulta con RANK() over (order By..) que le asigne a cada jugador su posición en el leaderboard global por mayor puntaje, manejando empates. Luego escribe una segunda consulta que logre el mismo resultado usando una subconsulta correlacionada (sin RANK)

1

Pregunta 2 Jugadores fantasma anti-join
Consigna (~~[illegible]~~ código en phpMyAdmin
Escribe una consulta que encuentre todos los jugadores que no han obtenido ningún item (no aparecen en inventario_items), usando left join is null

Pregunta 3. Vista restringida y principios de ~~[illegible]~~ menor privilegio
Consigna (código en phpMyAdmin)
El usuario analista necesita monitorear la actividad de jugadores, pero no debe poder ver la columna ~~[illegible]~~ (dato sensible)
Crea una view que excluya email, otorga select sobre la vista a analista y revoca el select directo que tenía sobre la tabla jugadores

Pregunta 4 provocar deadlock real forzando 'orden[illegible]')
consigna (codigo phpMyadmin)

abre dos pestañas de phpMyadmin en la pestaña A: start transaction, update jugadores set monedas = monedas - 10 where jugador id = 2; y luego intenta actualizar también el jugador 1. En la pestaña B, intenta actualizar también el jugador 2 provoca el error 1213 (el deadlock)

Pregunta 5 Condición de carrera sin for update
consigna (codigo en phpMyadmin):
Toma el procedimiento Comprar item unico y crea una copia modificada que quite la clausula for update del select. Ejecuta el mismo experimento de dos compras simultaneas (script o dos sesion ?) con esta version insegura.

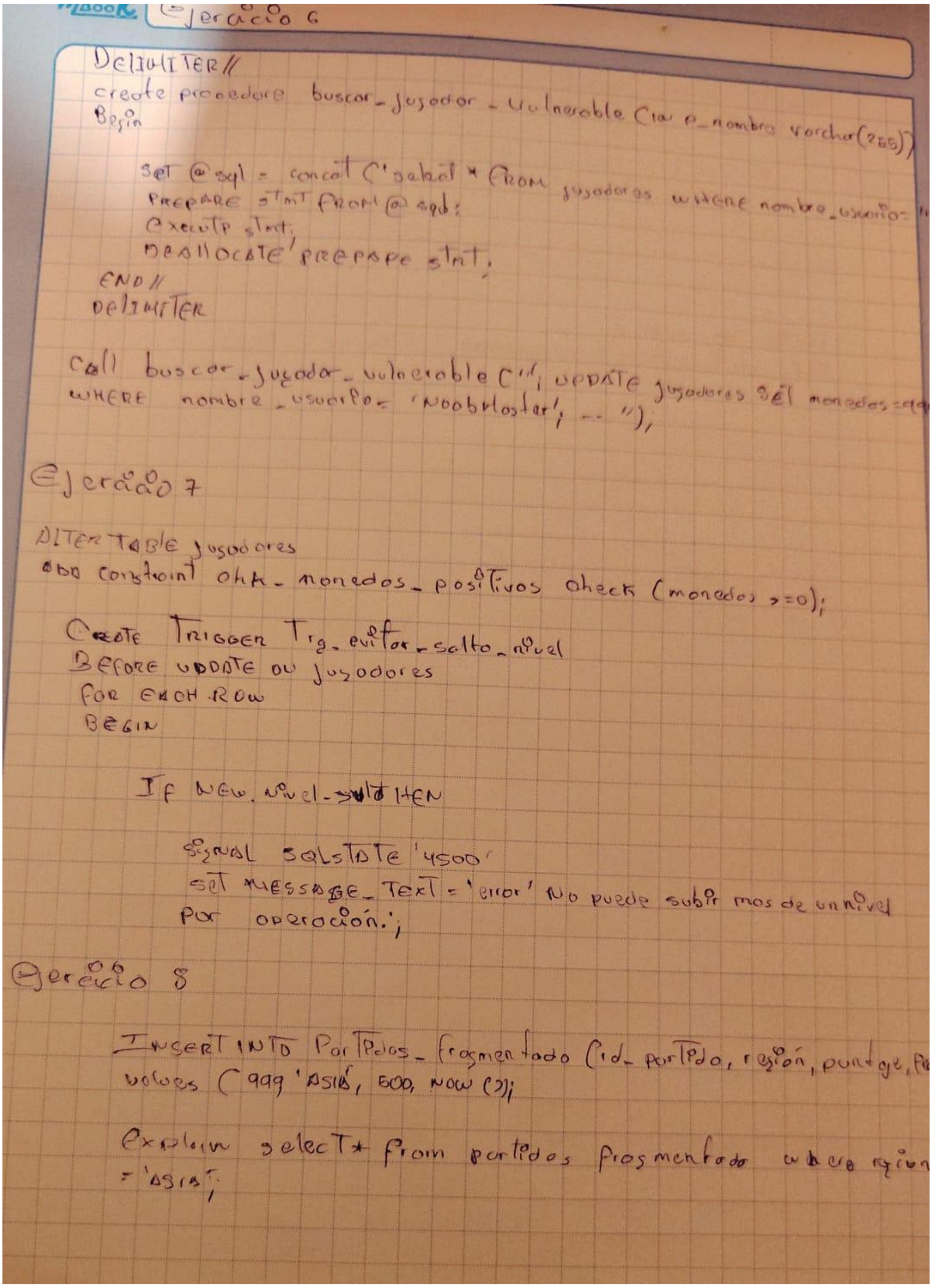
- Pregunta 6 Inyección con sentencia aplicadas (stacked queries)
  Consigna (codigo en phpMyAdmin
  Usando PREPARE/EXECUTE para simular como un backend mal hecho concatenaría texto, costruye el payload; UPDATE Jugadores SET monedas = 999999 WHERE nombre_usuario = 'NoobMaster' -- contra una consulta dinámica concatenada, y luego el mismo payload contra buscar jugador seguro.

- Pregunta 7 Anti-trampas con CHCK vs. TRIGGER
  consigna (códig en phpMyAdmin)

Escribe una restricción CHCK que inpida que moneda sea negativo, y además un TRIGGER BEFORE UPDATE que rechace (SIGNAL SQLSTATE '45000') cualquier intento de subic el nivel de un jugador en mas de 1 por operación.

Sesión 5 - Fragmentación y carga masiva

Pregunta 8 Agregar una participacion sin perder datos y probar partition pruning
consigna (codigo en php MyAdmin),
Sobre Partidas_Fragmentados, agrega una tercera partición p_asia VALUES IN ('ASIA') usando ALTER TABLE... ADD PARTITION, inserta un registro de prueba con region = 'ASIA', y corre EXPLAIN fitrado por esa región.



Ejercicio 6

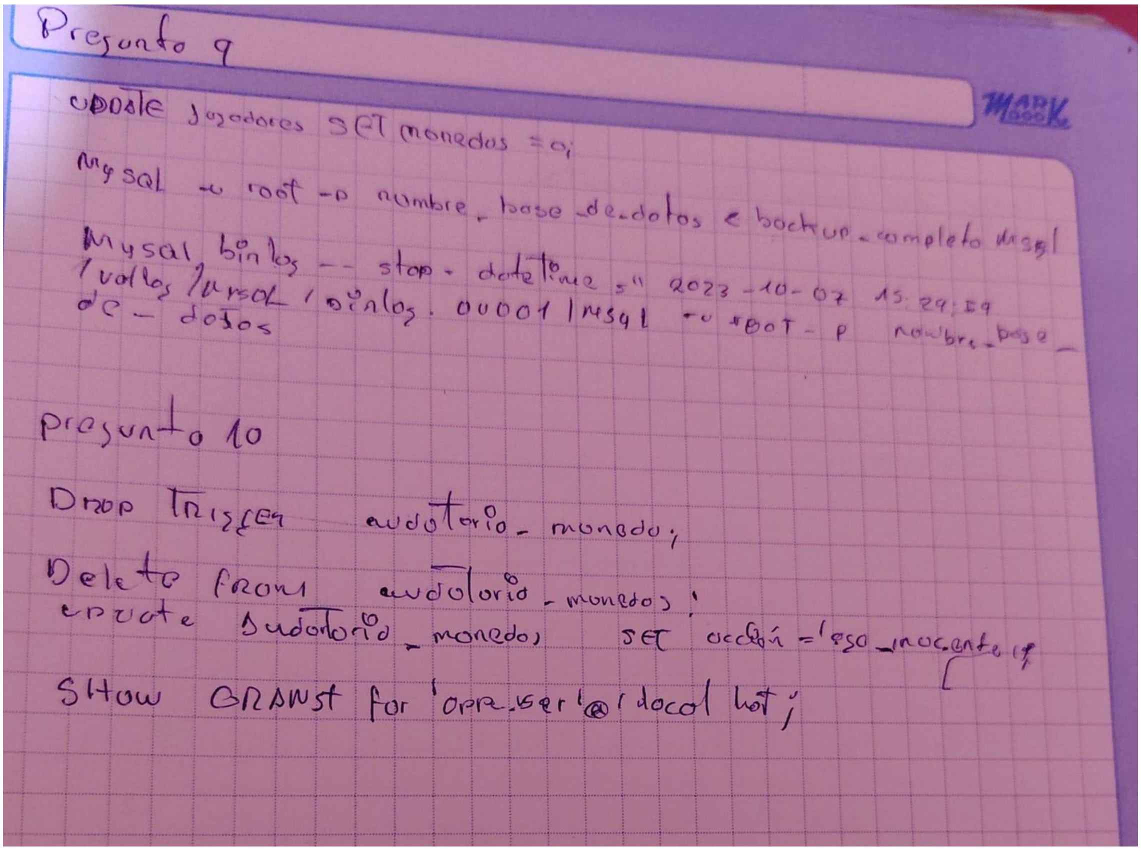
```
DELIMITER//
create procedure buscar_jugador_vulnerable (in p_nombre varchar(255))
Begin

    set @sql = concat ('select * FROM jugadores WHERE nombre_usuario= "
    PREPARE stmt FROM @sql;
    execute stmt;
    DEALLOCATE PREPARE stmt;
END//
DELIMITER

call buscar_jugador_vulnerable ("; UPDATE jugadores SET monedas=99
WHERE nombre_usuario= 'NoobMaster'; -- ");
```

Ejercicio 7

```
ALTER TABLE jugadores
add constraint chk_monedas_positivos check (monedas >=0);

 Create TRIGGER Trg_evitar_salto_nivel
 BEFORE UPDATE ON jugadores
 FOR EACH ROW
 BEGIN

     If NEW.nivel-[illegible] THEN

         signal SQLSTATE '4500'
         set MESSAGE_TEXT = 'error' No puede subir mas de un nivel
         por operación.';
```

Ejercicio 8

```
    INSERT INTO Partidos_fragmentado (id_partido, región, puntaje, fe
    values (999 'ASIA', 500, NOW ();

    explain selecT* from partidos fragmentado where region
    = 'ASIA';
```



Pregunta 9

```
UPDATE jugadores SET monedas = 0;
mysql -u root -p nombre_base_de_datos < backup_completo.mysql
mysqlbinlog --stop-datetime="2023-10-07 15:29:59
/var/log/mysql/binlog.00001 | mysql -u root -p nombre_base_
de_datos
```

pregunta 10

```
Drop TRIGGER auditoria_moneda;
Delete FROM auditoria_monedas;
update auditoria_monedas SET acción = 'eso_inocente';
SHOW GRANST for 'app_user'@'localhost';
```